

Cy Critical Runway

Modélisation et vérification d'un système d'atterrissage critique

Rapport de projet en Akka Typed,
Scala, réseau de Pétri et LTL

Établissement : CY Tech

Auteurs : Fares Ben Mabrouk
Maxime CLEMENT
Mehdi Boulaich
Nassym Ben Abdallah
Ismail Aitlaalim

Date : 12 avril 2026

Résumé. Dans ce projet, nous avons conçu et vérifié un système distribué critique de gestion d'atterrissage à l'aide d'Akka Typed et Scala. Le système repose sur trois acteurs principaux — *Avion*, *Tour de contrôle* et *Piste* — et met en évidence un problème central de concurrence autour d'une ressource critique unique. Nous avons ensuite construit un modèle de réseau de Pétri cohérent avec le comportement principal du code, afin d'explorer l'espace d'états, de contrôler des invariants métier et de formaliser plusieurs propriétés de sûreté et de vivacité en LTL. Le rapport présente nos choix de modélisation, l'architecture retenue, la stratégie de validation par tests, ainsi que l'apport du modèle formel dans l'analyse du système.

Table des matières

1	Introduction	1
2	Objectifs du projet	1
3	Cadre théorique et état de l'art	2
3.1	Vérification formelle des systèmes critiques	2
3.2	Pourquoi les réseaux de Pétri sont adaptés ici	2
3.3	Articulation entre Akka Typed, Pétri et LTL	3
3.4	Positionnement bibliographique	3
4	Architecture du système Akka / Scala	3
4.1	Vue générale	3
4.2	États métier	4
4.2.1	États d'un avion	4
4.2.2	États de la piste	4
4.3	Messages principaux	5
4.4	Gestion de la concurrence	5
4.5	Supervision	5
5	Traduction en réseau de Pétri	6
5.1	Principe de modélisation	6
5.2	Correspondance entre le code et les transitions	6
5.3	Intérêt du modèle formel	7
6	Propriétés structurelles et invariants métier	7
6.1	Exclusion mutuelle sur la piste	7
6.2	Unicité de l'état de la piste	7
6.3	Conservation du nombre d'avions	7
6.4	Pas d'autorisation sans réservation	7
6.5	Progression correcte vers le sol	7
7	Formalisation LTL	7
7.1	Pourquoi utiliser la LTL	8
7.2	Propositions atomiques	8

7.3	Propriétés de sûreté	8
7.4	Propriétés de vivacité	9
7.5	Positionnement méthodologique	9
7.6	Portée réelle de la vérification	10
8	Comparaison entre simulation Akka et modèle formel	10
8.1	Principe de la comparaison	10
8.2	Lecture comparée d'un scénario nominal	11
8.3	Ce que montre réellement cette comparaison	11
9	Implémentation des tests	12
9.1	Objectif des tests	12
9.2	Jeu de tests réalisé	12
9.3	Commentaires sur la validation	12
10	Analyseur formel et résultats	13
10.1	Résultats établis directement	13
10.2	Interprétation des résultats	13
10.3	Apport de l'analyseur dans le projet	13
11	Limites du projet	14
12	Bilan critique	14
13	Conclusion	14
	Bibliographie de référence	15

1 Introduction

Les systèmes distribués critiques apparaissent dans de nombreux domaines : transport, finance, télécommunications ou encore industrie. Dans ce type de systèmes, une erreur de coordination peut produire un comportement dangereux ou incohérent. Le sujet de ce projet nous a amenés à modéliser puis à vérifier un système critique distribué en nous appuyant sur Akka, Scala, les réseaux de Pétri et une formalisation en logique temporelle LTL.

Nous avons choisi de travailler sur un scénario d'atterrissage aéroportuaire. Ce choix nous a paru particulièrement pertinent pour plusieurs raisons :

- la **ressource critique** est immédiate à identifier : la piste ;
- les **interactions concurrentes** apparaissent naturellement dès que plusieurs avions demandent l'atterrissage ;
- les **propriétés métier** sont claires à formuler : pas de double occupation de piste, pas d'atterrissage sans autorisation, progression correcte jusqu'au sol ;
- le **lien entre implémentation et vérification formelle** est direct.

Notre objectif n'était pas de reproduire toute la complexité d'un système aéroportuaire réel, mais de construire un modèle suffisamment riche pour faire apparaître de vraies interactions critiques, tout en restant assez clair pour être implémenté, testé et vérifié formellement. Nous avons donc privilégié une architecture simple, mais rigoureuse, dans laquelle chaque choix de modélisation reste directement exploitable à la fois dans le code Akka et dans le réseau de Pétri.

Dans cette logique, nous avons choisi de nous concentrer sur :

- une architecture d'acteurs lisible et cohérente ;
- des invariants métier clairement identifiés ;
- une implémentation Akka Typed fidèle au comportement attendu ;
- un modèle de réseau de Pétri aligné avec les mécanismes critiques du code ;
- une validation combinant tests et exploration d'espace d'états.

2 Objectifs du projet

Le projet doit répondre à quatre objectifs techniques principaux :

1. **modéliser** un système distribué critique à l'aide d'acteurs Akka ;
2. **identifier** les interactions concurrentes et les états sensibles ;
3. **traduire** ce fonctionnement en un modèle formel de réseau de Pétri ;
4. **vérifier** des propriétés structurelles, métier et temporelles.

Dans notre cas, cela revient à garantir au minimum les points suivants :

- un seul avion peut utiliser la piste à un instant donné ;
- un avion ne peut pas atterrir sans autorisation ;
- les demandes concurrentes sont traitées sans incohérence ;
- les avions finissent par atteindre l'état *Au sol* dans le scénario nominal ;
- le modèle formel ne présente pas de deadlock non terminal.

3 Cadre théorique et état de l'art

Cette section nous permet de replacer notre projet dans le cadre demandé par le sujet, en présentant brièvement les notions sur lesquelles nous nous appuyons : la vérification formelle, les réseaux de Pétri et la logique temporelle LTL. L'idée n'est pas de faire un état de l'art exhaustif, mais de montrer clairement pourquoi ces outils sont pertinents dans notre cas et comment ils s'articulent avec les choix que nous avons faits dans le projet.

3.1 Vérification formelle des systèmes critiques

La vérification formelle regroupe les méthodes qui permettent de raisonner rigoureusement sur le comportement d'un système, non plus seulement par tests empiriques, mais à l'aide d'un modèle mathématique ou logique. Dans les systèmes critiques, cette démarche est particulièrement utile, car une exécution correcte sur quelques cas ne suffit pas à garantir que *tous* les comportements possibles respectent les contraintes de sûreté.

Dans notre projet, la vérification formelle intervient à deux niveaux :

- au niveau de la **spécification**, avec l'écriture de propriétés de sûreté et de vivacité en LTL ;
- au niveau de la **modélisation**, avec la traduction du système Akka vers un réseau de Pétri et l'exploration de son espace d'états.

Cette approche est classique dans l'étude des systèmes concurrents : on part d'une implémentation ou d'une architecture logicielle, puis on construit un modèle plus abstrait permettant d'analyser les interblocages, les séquences d'exécution et les invariants.

3.2 Pourquoi les réseaux de Pétri sont adaptés ici

Les réseaux de Pétri sont particulièrement adaptés à la modélisation des systèmes distribués et concurrents, car ils permettent de représenter explicitement :

- des **états** sous forme de places ;
- des **changements d'état** sous forme de transitions ;
- des **ressources partagées** via la présence ou l'absence de jetons ;
- des **séquences concurrentes** et des conflits d'accès à une ressource.

Dans notre projet, la piste constitue précisément une ressource critique partagée. Le réseau de Pétri fournit donc un cadre naturel pour représenter l'exclusion mutuelle, l'attente, la réservation puis l'occupation effective de cette ressource.

Le choix d'un modèle de Pétri est également cohérent avec l'objectif du sujet, qui insiste sur l'analyse de propriétés structurelles comme l'absence de deadlocks, la cohérence des transitions et la conservation d'invariants métier.

3.3 Articulation entre Akka Typed, Pétri et LTL

Akka Typed apporte une modélisation opérationnelle claire du système distribué : les acteurs échangent des messages, évoluent localement et coopèrent pour gérer l'accès à la piste. Cette couche représente le **comportement exécutable** du projet.

Le réseau de Pétri constitue quant à lui une **abstraction formelle** de ce comportement. Il ne cherche pas à reproduire chaque détail d'implémentation, mais à capturer les états significatifs et les transitions critiques nécessaires au raisonnement.

Enfin, la LTL joue ici le rôle de **langage de spécification**. Elle permet d'exprimer proprement ce que nous attendons du système sur la durée : certaines situations ne doivent jamais arriver, d'autres doivent finir par se produire. Le projet ne met pas en œuvre un model checker LTL complet, mais il s'appuie sur la LTL pour définir précisément les propriétés qui guident l'analyse.

3.4 Positionnement bibliographique

Notre travail s'inscrit dans une base théorique volontairement ciblée :

- la documentation officielle d'Akka Typed pour les aspects acteurs, supervision et communication typée ;
- les travaux classiques sur les réseaux de Pétri pour les propriétés structurelles et l'analyse des comportements concurrents ;
- les ouvrages de logique en informatique pour la formalisation en logique temporelle.

Ce cadrage nous permet de rester cohérents avec le niveau attendu par le module tout en reliant clairement l'implémentation logicielle, la modélisation formelle et la formulation des propriétés.

4 Architecture du système Akka / Scala

4.1 Vue générale

Le système repose sur trois acteurs principaux :

- **Avion** : représente un avion individuel et son cycle de vie ;
- **TourDeContrôle** : centralise les demandes, gère la file d'attente FIFO et coordonne l'accès à la piste ;

- **Piste** : représente la ressource critique et impose une exclusion mutuelle stricte.

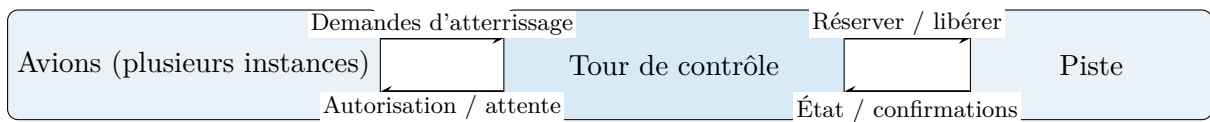


FIGURE 1 – Architecture logique simplifiée du système distribué.

Nous avons retenu cette architecture parce qu'elle nous permet de faire apparaître clairement les responsabilités de chaque acteur tout en gardant un système suffisamment compact pour être analysé formellement. Ce découpage nous a aussi aidés à relier plus facilement le comportement du code aux états et transitions du réseau de Pétri.

4.2 États métier

4.2.1 États d'un avion

Chaque avion suit le cycle de vie suivant :

- `AvionEnVol`
- `AvionEnAttente`
- `AvionAutorise`
- `AvionAtterrissageEnCours`
- `AvionAuSol`

Ce découpage est volontairement simple. Il nous permet de représenter le comportement du système sans multiplier artificiellement les états.

4.2.2 États de la piste

La piste possède trois états exclusifs :

- `PisteLibre`
- `PisteReservee`
- `PisteOccupee`

L'introduction de l'état *Réservée* est importante : elle distingue le moment où la piste est attribuée à un avion du moment où elle est effectivement occupée physiquement.

4.3 Messages principaux

TABLE 1 – Messages principaux du protocole.

Message	Émetteur principal	Rôle
DemandeAtterrissage	Avion	Demande d'accès au système d'atterrissage
AutorisationAccordee	Tour	Autorisation d'atterrir pour un avion donné
MiseEnAttente	Tour	Placement d'un avion dans la file d'attente
ReserverPour	Tour	Demande de réservation de piste
ReservationAccordee	Piste	Confirmation qu'une réservation a été acceptée
Occuper	Avion	Occupation effective de la piste
FinAtterrissage	Avion	Notification de fin de manœuvre à la tour
Liberer	Tour	Demande de libération de la piste
LiberationConfirnee	Piste	Confirmation que la piste est redevenue libre
EtatPisteActuel	Piste	Réponse de synchronisation à la tour

4.4 Gestion de la concurrence

La concurrence est gérée par deux mécanismes complémentaires :

- la **piste** impose une exclusion mutuelle stricte ;
- la **tour de contrôle** maintient une **file FIFO** lorsque la piste n'est pas disponible.

Lorsqu'un avion demande l'atterrissage :

1. si la piste est libre, la tour demande une réservation ;
2. une fois la réservation confirmée, l'avion reçoit l'autorisation ;
3. l'avion occupe la piste et réalise sa manœuvre ;
4. à la fin, la piste est libérée ;
5. si des avions sont en attente, le premier de la file est traité.

4.5 Supervision

Nous avons également intégré une supervision simple de la tour de contrôle. Lorsqu'un crash simulé est déclenché, la tour redémarre puis se resynchronise avec l'état courant de la piste. Nous avons volontairement gardé cette reprise à un niveau simple : elle montre concrètement un mécanisme de redémarrage cohérent, sans alourdir inutilement l'architecture avec une persistance complète de tout l'état métier.

5 Traduction en réseau de Pétri

5.1 Principe de modélisation

Le réseau de Pétri reprend les états essentiels du système afin de représenter, de manière abstraite, les évolutions possibles du scénario d'atterrissage.

Les places principales du modèle sont les suivantes :

- EnVol
- EnAttente
- Autorise
- AtterrissageEnCours
- AuSol
- DansFile
- PisteLibre
- PisteReservee
- PisteOccupee

Pour la partie formelle, nous avons choisi un modèle de réseau de Pétri agrégé. Ce choix nous permet de représenter les états essentiels du protocole d'atterrissage et d'explorer l'espace d'états de manière plus lisible, tout en restant fidèle à la dynamique globale du système Akka. L'objectif n'était pas de reproduire chaque détail d'implémentation, mais de capturer les comportements critiques nécessaires à la vérification.

5.2 Correspondance entre le code et les transitions

TABLE 2 – Correspondance entre transitions du réseau de Pétri et comportement Akka.

Transition	Effet sur le marquage	Signification dans le code
T1	EnVol $\rightarrow$ EnAttente	émission de DemandeAtterrissage
T2	EnAttente + PisteLibre $\rightarrow$ Autorise + PisteReservee	réservation confirmée puis autorisation envoyée
T3	EnAttente $\rightarrow$ DansFile	avion placé en attente lorsque la piste n'est pas libre
T4	Autorise + PisteReservee $\rightarrow$ AtterrissageEnCours + PisteOccupee	occupation effective de la piste
T5	AtterrissageEnCours $\rightarrow$ AuSol	fin de manœuvre de l'avion
T6	PisteOccupee $\rightarrow$ PisteLibre	libération de la piste
T7	DansFile + PisteLibre $\rightarrow$ Autorise + PisteReservee	traitement FIFO du prochain avion

5.3 Intérêt du modèle formel

Le réseau de Pétri nous permet de raisonner sur :

- les **séquences possibles** d'exécution ;
- la **cohérence structurelle** des états ;
- l'**absence de deadlocks** dans le scénario nominal ;
- l'atteignabilité d'un **état terminal correct** ;
- la vérification d'invariants qui seraient plus difficiles à garantir uniquement par lecture du code.

6 Propriétés structurelles et invariants métier

Nous avons retenu les propriétés suivantes comme étant les plus importantes pour le système.

6.1 Exclusion mutuelle sur la piste

Deux avions ne doivent jamais occuper la piste en même temps. C'est la propriété de sûreté centrale du projet.

6.2 Unicité de l'état de la piste

La piste doit toujours être dans exactement un état parmi : *Libre*, *Réservée*, *Occupée*. Cela évite les situations incohérentes où la piste serait simultanément considérée comme libre et occupée.

6.3 Conservation du nombre d'avions

Le modèle ne doit ni perdre ni créer d'avions. À tout instant, la somme des avions présents dans les différents états doit rester constante.

6.4 Pas d'autorisation sans réservation

Un avion ne peut pas être dans l'état *Autorisé* si la piste n'a pas été préalablement réservée pour lui.

6.5 Progression correcte vers le sol

Dans le scénario nominal, un avion autorisé doit finir par atteindre l'état *Au sol*, et la piste doit ensuite être libérée.

7 Formalisation LTL

7.1 Pourquoi utiliser la LTL

La logique temporelle linéaire (LTL) permet de formaliser les garanties attendues d'un système concurrent. Dans notre projet, elle est utilisée comme **langage de spécification** : elle ne sert pas à implémenter un model checker complet, mais à écrire proprement les propriétés que notre système doit respecter.

Nous ne prétendons donc pas avoir model-checké directement toutes les formules LTL écrites dans ce rapport. En revanche, ces formules servent de référence conceptuelle pour orienter :

- les invariants contrôlés dans le modèle ;
- les scénarios validés par tests ;
- les propriétés observées lors de l'exploration d'espace d'états.

Autrement dit, la LTL joue ici un rôle de **spécification rigoureuse** et de **guide méthodologique**.

7.2 Propositions atomiques

On introduit les propositions atomiques suivantes :

- L : la piste est libre ;
- R : la piste est réservée ;
- O : la piste est occupée ;
- A : au moins un avion est autorisé ;
- C : au moins un avion est en cours d'atterrissage ;
- T : tous les avions sont au sol.

Pour un avion donné i , on note aussi :

- Dem_i : l'avion i a émis une demande ;
- $Wait_i$: l'avion i est en attente ;
- $Auth_i$: l'avion i est autorisé ;
- $Ground_i$: l'avion i est au sol.

7.3 Propriétés de sûreté

S1 – la piste ne peut pas être dans deux états à la fois.

$$G\neg(L \wedge R) \wedge G\neg(L \wedge O) \wedge G\neg(R \wedge O)$$

Cette propriété exprime l'unicité de l'état de la piste.

S2 – une autorisation implique une réservation préalable.

$$G(A \rightarrow R)$$

Cette formule traduit le fait qu'un avion ne peut être autorisé à atterrir que si la piste a été réservée.

S3 – un atterrissage en cours implique une piste occupée.

$$G(C \rightarrow O)$$

Cette propriété empêche toute incohérence entre l'état avion et l'état de la ressource critique.

7.4 Propriétés de vivacité

V1 – toute occupation de piste finit par se terminer.

$$G(O \rightarrow FL)$$

Autrement dit, une piste occupée doit redevenir libre à terme.

V2 – tout scénario nominal finit par atteindre un état terminal correct.

$$FT$$

Cette propriété formalise le fait qu'à terme, tous les avions du scénario atteignent l'état *Au sol*.

V3 – toute demande reçoit finalement une réponse.

$$G(Dem_i \rightarrow F(Wait_i \vee Auth_i))$$

Un avion qui demande l'atterrissage ne doit pas rester sans retour du système.

V4 – tout avion autorisé finit par atteindre le sol.

$$G(Auth_i \rightarrow FGround_i)$$

7.5 Positionnement méthodologique

Il est important de préciser que notre travail n'implémente pas un **model checker LTL générique**. Nous avons adopté une démarche plus légère et adaptée au projet :

1. formalisation des propriétés en LTL ;
2. abstraction du système en réseau de Pétri ;

3. exploration exhaustive de l'espace d'états ;
4. contrôle des invariants, des deadlocks et de l'atteignabilité des états terminaux ;
5. confrontation avec les scénarios observés dans l'implémentation Akka.

Cette démarche reste cohérente avec le cadre du sujet, à condition de bien distinguer :

- ce qui est **spécifié** en LTL ;
- ce qui est **vérifié directement** par l'analyseur et les tests ;
- ce qui est **soutenu indirectement** par la cohérence entre modèle formel et implémentation.

7.6 Portée réelle de la vérification

Dans la version actuelle du projet, les résultats obtenus permettent d'établir directement :

- le respect d'invariants structurels sur les états explorés ;
- l'absence de deadlock non terminal détecté dans le scénario nominal ;
- l'atteignabilité d'un état terminal correct où tous les avions sont au sol ;
- la cohérence globale entre les principales transitions Akka et le réseau de Pétri.

En revanche, nous ne prétendons pas démontrer au sens fort toutes les propriétés LTL sur tous les chemins possibles comme le ferait un model checker dédié. Les formules LTL doivent donc être comprises comme une **spécification formelle de référence**, dont plusieurs conséquences sont ensuite vérifiées par exploration d'états et par validation expérimentale.

8 Comparaison entre simulation Akka et modèle formel

Le sujet demande non seulement de simuler le système distribué, mais aussi de comparer le comportement observé avec le modèle formel. Cette comparaison est importante dans notre démarche, car nous ne voulions pas que le réseau de Pétri soit seulement un schéma théorique ajouté à côté du code. Au contraire, nous avons cherché à construire un modèle formel qui reste cohérent avec les mécanismes critiques réellement présents dans l'implémentation Akka : demande d'atterrissage, réservation, occupation de la piste, attente et libération.

8.1 Principe de la comparaison

La simulation Akka fait apparaître une suite concrète d'événements : demandes d'atterrissage, mise en attente, réservation, occupation de la piste, fin de manœuvre puis libération. Le réseau de Pétri reprend cette logique sous forme de transitions abstraites. L'objectif n'est pas d'obtenir une identité parfaite message par message, mais une correspondance cohérente entre :

- les **états métier** observés dans le code ;
- les **changements d'état** représentés dans le réseau ;

— les **propriétés** que l'on cherche à vérifier.

8.2 Lecture comparée d'un scénario nominal

Le tableau suivant illustre cette correspondance sur un scénario typique avec plusieurs avions et une seule piste.

Étape	Observation côté Akka	Transition / état côté Pétri	Intérêt pour la vérification
1	Un avion en vol envoie DemandeAtterrissage à la tour	$T1 : EnVol \rightarrow EnAttente$	Le système prend en charge une demande explicite et fait entrer l'avion dans le processus
2	La tour constate que la piste est libre et demande une réservation	préparation de $T2$ avec présence de PisteLibre	Vérifie que l'autorisation n'est pas donnée sans disponibilité de la ressource
3	La piste confirme la réservation et la tour envoie AutorisationAccordee	$T2 : EnAttente + PisteLibre \rightarrow Autorise + PisteReservee$	Lie l'autorisation à la réservation préalable
4	L'avion autorisé envoie Occuper et commence sa manœuvre	$T4 : Autorise + PisteReservee \rightarrow AtterrissageEnCours + PisteOccupee$	Vérifie la cohérence entre l'état avion et l'état de la piste
5	Si un autre avion demande pendant ce temps, il reçoit MiseEnAttente	$T3 : EnAttente \rightarrow DansFile$	Représente le traitement concurrent sans double occupation
6	L'avion termine et la tour envoie Liberer à la piste	$T5$ puis $T6$	Vérifie la progression vers AuSol et le retour de la piste à l'état libre
7	Si des avions attendent, la tour traite le suivant selon l'ordre FIFO	$T7 : DansFile + PisteLibre \rightarrow Autorise + PisteReservee$	Représente la reprise ordonnée de la file d'attente

8.3 Ce que montre réellement cette comparaison

Cette comparaison nous permet d'affirmer que les **mécanismes critiques du code** sont bien repris dans le modèle formel :

- l'exclusion mutuelle autour de la piste ;
- la distinction entre réservation et occupation ;
- la gestion de l'attente lorsque la ressource n'est pas disponible ;
- la progression nominale jusqu'à l'état terminal.

Elle montre aussi les limites de l'abstraction choisie. Le réseau de Pétri utilisé ici est **agrégé** : il capture correctement la dynamique globale, mais il ne distingue pas chaque avion par son identité propre dans le marquage. En conséquence, certaines propriétés fines, comme l'ordre FIFO au niveau individuel, sont davantage validées par les **tests Akka** que par la seule structure du réseau.

9 Implémentation des tests

9.1 Objectif des tests

Les tests servent à valider le comportement concret du système Akka. Ils complètent la partie formelle en montrant que l'implémentation suit bien les propriétés attendues.

9.2 Jeu de tests réalisé

Dans la version finale du projet, nous avons mis en place **8 tests** :

N°	Test	Intérêt
1	Autorisation immédiate si piste libre	Vérifie le scénario nominal le plus simple
2	Mise en attente d'un deuxième avion si piste occupée	Vérifie la gestion d'une demande concurrente
3	Respect de l'ordre FIFO	Vérifie la cohérence de la file d'attente
4	Ignorer une libération avec un mauvais identifiant	Vérifie la robustesse de la ressource critique
5	Retour à l'état libre quand la file est vide	Vérifie la libération correcte de la piste
6	Resynchronisation après crash de la tour	Vérifie la supervision simple et la reprise partielle
7	Scénario réel avec deux avions	Vérifie le cycle complet avec de vrais acteurs Avion
8	Scénario réel avec trois avions	Vérifie la progression logique du système sur un cas plus riche

9.3 Commentaires sur la validation

L'intérêt du jeu de tests est double :

- vérifier les **règles métier locales** ;
- valider le **fonctionnement bout en bout** du système concurrent.

Le choix d'inclure des tests avec de vrais avions est particulièrement important, car il permet de vérifier non seulement la logique de la tour et de la piste, mais aussi l'enchaînement réel des messages dans un scénario complet.

10 Analyseur formel et résultats

L'analyseur du réseau de Pétri effectue une exploration en largeur (BFS) de l'espace d'états à partir du marquage initial.

Dans notre modèle :

- le nombre d'avions est fixé à 3 ;
- chaque transition modifie le marquage conformément au protocole Akka abstrait ;
- les invariants sont contrôlés sur chaque état exploré ;
- les états sans successeur sont examinés afin de détecter un éventuel deadlock non terminal.

10.1 Résultats établis directement

Dans le cadre du **modèle agrégé** et du **scénario nominal** exploré, l'analyse permet d'établir directement les points suivants :

- aucune violation des invariants définis n'est détectée sur les états parcourus ;
- aucun deadlock non terminal n'est observé dans l'espace d'états exploré ;
- un état terminal correct où tous les avions sont au sol est atteignable ;
- la piste redevient libre à l'issue des scénarios terminaux atteints.

10.2 Interprétation des résultats

Ces résultats sont importants, car ils montrent que le modèle ne produit pas de séquence nominale incohérente et qu'il existe bien une progression complète du système vers un état final correct.

En revanche, nous formulons volontairement ces conclusions avec prudence :

- l'analyseur ne constitue pas un model checker LTL générique ;
- certaines propriétés de vivacité sont **approchées** à partir de l'atteignabilité et de la cohérence des transitions, plutôt que prouvées au sens temporel fort sur tous les chemins ;
- le caractère agrégé du modèle limite l'analyse de propriétés dépendant d'identités individuelles.

10.3 Apport de l'analyseur dans le projet

L'intérêt de l'analyseur est donc double :

- fournir une **vérification structurée** d'invariants et d'absence de deadlocks non terminaux ;
- renforcer la crédibilité du projet en montrant que le comportement concurrent implémenté dans Akka possède une **abstraction formelle exploitable**.

Ainsi, l'analyseur ne remplace pas les tests ni un outil complet de model checking, mais il constitue une étape pertinente de validation formelle, cohérente avec le niveau et les objectifs du projet.

11 Limites du projet

Comme tout modèle, notre projet repose sur des choix de périmètre. Nous avons préféré concentrer l'effort sur les mécanismes critiques les plus importants plutôt que de multiplier les cas particuliers. Les limites suivantes ne correspondent donc pas à des oublis, mais à des simplifications assumées pour garder un système à la fois cohérent, vérifiable et défendable dans le cadre du sujet.

- Nous avons modélisé une seule piste, afin de concentrer l'analyse sur une ressource critique unique et de rendre la vérification plus lisible.
- La supervision de la tour reste volontairement partielle : après redémarrage, la tour se re-synchronise avec la piste, mais ne reconstruit pas toute son histoire métier.
- Le réseau de Pétri est agrégé : il représente fidèlement la dynamique globale du système, mais pas l'identité individuelle de chaque avion dans le marquage.
- La LTL est utilisée comme langage de spécification des propriétés ; nous ne mettons pas en œuvre un model checker temporel complet.

12 Bilan critique

Le principal point fort de notre projet est la cohérence entre les différentes couches que nous avons construites : l'architecture Akka, les états métier, la gestion de la ressource critique, les tests et le modèle formel. Cette cohérence nous a permis de ne pas traiter le réseau de Pétri comme un simple ajout théorique, mais comme une véritable abstraction du comportement central du système.

Par ailleurs, plusieurs choix nous semblent particulièrement importants :

- la séparation claire entre `Avion`, `TourDeControle` et `Piste` ;
- l'introduction de l'état `PisteReservee`, qui affine le modèle de concurrence ;
- la file d'attente FIFO, qui donne un comportement cohérent côté implémentation ;
- l'articulation entre tests, réseau de Pétri et formalisation LTL.

Avec ce projet, nous montrons qu'il est possible de partir d'une implémentation concurrente concrète, d'en extraire les mécanismes critiques, puis de les formaliser de manière suffisamment rigoureuse pour raisonner sur leur correction.

13 Conclusion

Au cours de ce projet, nous avons conçu, implémenté et analysé un système distribué critique de gestion d'atterrissage fondé sur trois acteurs principaux : les avions, la tour de contrôle et la piste.

Cette architecture nous a permis de mettre en œuvre une gestion concurrente claire autour d'une ressource critique, tout en gardant un système lisible et testable.

Nous avons ensuite abstrait ce comportement à l'aide d'un réseau de Pétri afin d'explorer les états possibles du système et de vérifier plusieurs propriétés importantes de sûreté et de vivacité. La formalisation en LTL nous a servi de cadre pour exprimer proprement ces garanties et structurer notre raisonnement.

Ce projet nous a surtout permis de relier plusieurs dimensions complémentaires : l'implémentation, la modélisation, la spécification formelle et la validation. C'est cette articulation entre code exécutable, réseau de Pétri, propriétés temporelles et tests qui constitue, selon nous, le cœur du travail réalisé.

Bibliographie de référence

- Documentation officielle *Akka Typed* : acteurs typés, supervision, communication par messages et tests d'acteurs.
- T. Murata, *Petri Nets : Properties, Analysis and Applications*. Référence classique sur les propriétés structurelles et comportementales des réseaux de Pétri.
- M. Huth, M. Ryan, *Logic in Computer Science*. Ouvrage de référence pour la logique temporelle, la vérification et le raisonnement sur les systèmes.
- K. Jensen, L. M. Kristensen, *Coloured Petri Nets*. Référence utile pour le positionnement général des réseaux de Pétri dans la modélisation des systèmes concurrents.
- Cours, consignes du module et supports pédagogiques du projet, utilisés pour cadrer le périmètre méthodologique retenu.